

scripts/check-commit-docs-exists.sh — User Guide

Last verified: 2026-09-19 (submodule-nested path resolution — a commit body describing a fix made inside a top-level OR nested submodule no longer needs to spell out the submodule prefix to resolve; fixes a false positive on commit f6ff59eb) **Inheritance:** HelixConstitution §11.4.x (commit-references-resolve mandate) + Lava §6.AD-debt closure (CM-COMMIT-DOCS-EXISTS)

Overview

Per-commit reference resolver. Scans commit message bodies for file path citations and verifies each one resolves to a real file at HEAD. A commit message that cites "evidence at .lava-ci-evidence/X.json" when X.json was never created (or was deleted later) is a §6.J spirit violation — the claim is unfalsifiable because the referenced evidence does not exist.

Usage

```
# Default — scan HEAD only
bash scripts/check-commit-docs-exists.sh

# Scan a range of commits
bash scripts/check-commit-docs-exists.sh HEAD~5..HEAD

# Scan all unpushed commits (recommended pre-push)
bash scripts/check-commit-docs-exists.sh '@{u}..HEAD'

# Env-var alternative (used by hermetic tests + the sweep wrapper)
LAVA_COMMIT_RANGE='HEAD~3..HEAD' bash scripts/check-commit-docs-exists.sh
LAVA_REPO_ROOT=/path/to/synthetic bash scripts/check-commit-docs-exists.sh
```

Exit codes

Code	Meaning
0	Every referenced path resolves (exact or fuzzy-basename) — and at least one commit message was actually read
1	At least one orphan reference (commit SHA + path printed to stderr), or the range resolved but selects zero commits
2	The range failed to resolve — git could not name the commits, so nothing was examined (added 2026-08-26)

What counts as a path reference

A token in a commit message body is considered a path reference when ALL of these hold:

1. It contains at least one / (directory separator).
2. It ends with one of the recognized extensions: md json sh kt go kts gradle xml yaml yml toml conf txt tsv.
3. It starts with one of the canonical project-root prefixes: docs/, scripts/, tests/, .lava-ci-evidence/, submodules/, constitution/, app/, core/, feature/, proxy/, lava-api-go/, tools/, buildSrc/, config/, keystores/, releases/, Upstreams/, .githubhooks/.

These three conditions together filter out version IDs (1.2.30-1050), package names that happen to contain dots, and URLs.

Whitelist conditions (intentional skips)

- **Backtick spans** (``like-this``) are stripped before regex match. Prose markup is not a citation.
- **~strikethrough~ spans** are stripped. The closure-mark convention intentionally references obsolete paths.
- **Blockquote lines** (lines starting with >) are dropped. \$6.L cycle bodies often quote operator messages verbatim where the original might reference now-renamed paths.
- **Glob/wildcard paths** containing * are skipped. They are intentional non-existent path patterns.

Fuzzy basename fallback

A prose reference like `feature/search_result/SearchPageState.kt` resolves the real file at `feature/search_result/src/main/kotlin/lava/search/result/SearchPageState.kt` via this fallback:

1. Exact path check first.
2. If miss: extract the leading directory (e.g. `feature`) + the basename (`SearchPageState.kt`).
3. `find <prefix-dir> -name <basename> excluding build/, .git/, .claude/.`
4. If 1+ matches found, treat as resolved.

The fallback prevents pedantic false-positives on human-written short-form paths while still catching genuinely-missing files.

Submodule-nested path resolution (added 2026-09-19)

A commit may legitimately describe a fix made **inside** a submodule — top-level (`submodules/helixqa`) or nested (a submodule-of-a-submodule, e.g. `submodules/helixqa/tools/opensource/docling`) — without spelling out the submodule's own path prefix in prose. Example forensic anchor: commit `f6ff59eb` cited `tests/data/uspto/sources/pftaps057006474.txt`, describing a CRLF fix made inside `submodules/helixqa`'s nested `docling` submodule, where the file actually lives at `submodules/helixqa/tools/opensource/docling/tests/data/uspto/sources/pftaps057006474.txt`. The root-only existence check had no way to resolve that and flagged a real, resolvable reference as an orphan.

Resolution order per candidate path:

1. Exact path check at the repo root.
2. `.gitignore` skip (operator-local evidence).
3. **Submodule-nested resolution** — enumerate every submodule root, top-level AND nested, via `git submodule foreach --quiet --recursive 'echo "$displaypath"', and accept the candidate if <submodule-root>/<candidate> exists on disk.`
4. Fuzzy top-level basename fallback (unchanged).

This does **not** weaken the check for its real purpose: a path that resolves nowhere — not at the repo root, not inside any submodule at any depth — still fails. A candidate is only accepted when a real file exists at the joined path; there is no blanket "commit mentions a submodule" exemption.

§6.J anti-bluff falsifiability rehearsal

1. Create an empty git commit referencing a phantom file:

```
git commit --allow-empty -m "claim: see .lava-ci-evidence/  
sixth-law-incidents/2026-99-99-phantom.json"
```

2. Run `bash scripts/check-commit-docs-exists.sh` → expect exit 1 with the phantom path listed.
3. Either `git commit --amend` removing the reference, OR create the file, OR drop the commit.

The deliberate-phantom rehearsal proves the gate fires on real evidence gaps.

2026-08-26 update — §6.J corpus floor: "no commits in range" is no longer a pass

The defect

The gate collapsed three very different conditions into one `no commits in range '<range>'` – skipping message and **exit 0**:

Invocation	Old verdict
HEAD~5..HEAD in a repository with 1 commit	no commits in range — exit 0
HEAD~5..HEAKD (a typo in the ref)	no commits in range — exit 0
The same repository, range HEAD	VIOLATION: 2 orphan refs — exit 1

Only the third examined anything. The first two reported a clean gate having read **zero commit messages**.

This was not a theoretical corner. `scripts/verify-all-constitution-rules.sh` invokes this gate with a **hardcoded** `HEAD~5..HEAD`, so any checkout with fewer than six commits — a `--depth-limited` clone, a

worktree cut from a fresh branch, a CI-side shallow fetch — passed it silently, forever, while the same tree with the range spelled HEAD produced real violations.

The mechanism was `2>/dev/null` on the `git rev-list` call: git's **exit status** is the only thing that separates *"this ref cannot be resolved"* (128) from *"this range is valid and genuinely empty"* (0 with no output), and redirecting `stderr` discarded exactly that signal.

What the gate now refuses

Two distinct refusals, because the two causes have different remedies and a diagnosis that misstates its cause sends the reader to the wrong one:

Exit 2 — the range failed to resolve. git's own `stderr` and exit status are reported. The message states plainly that this is *not* an empty range: the range names a ref git cannot resolve — a typo, or a history too shallow for the offset. Three remedies are offered: fix the spelling, pass a range this history can satisfy (`LAVA_COMMIT_RANGE=HEAD`), or deepen a shallow clone with `git fetch --unshallow`.

Exit 1 — the range resolved but selects zero commits. The message states that the range is *valid* (git resolved it without error) and simply names an empty set — an `A..B` where `B` is an ancestor of `A`, or a self-range such as `HEAD..HEAD`. A gate that reads no commit message asserts nothing about whether commit bodies cite paths that exist, so this is a failure rather than a skip.

Both messages report `Examined: 0 commit message(s)` explicitly, so the corpus size is a stated fact rather than something the reader infers from the absence of output.

Consequence for callers

A shallow or fresh checkout that previously sailed through this gate will now fail it, with an actionable message. That is the intended outcome: the alternative is a gate whose verdict is a function of clone depth rather than of commit-body honesty. Where a narrow scope is genuinely wanted, name it explicitly — `LAVA_COMMIT_RANGE=HEAD` for the tip commit alone, or `'@{u}..HEAD'` for everything unpushed.

§6.J falsifiability rehearsal of the floor itself

1. `bash scripts/check-commit-docs-exists.sh 'HEAD~5..HEAKD'`
(deliberate typo) → expect **exit 2**, FAILED TO RESOLVE, and git's own error reproduced. Before this change: exit 0, reported as a skip.
2. `bash scripts/check-commit-docs-exists.sh 'HEAD..HEAD'` → expect **exit 1**, resolved but selects ZERO commits. Before this change: exit 0.
3. `bash scripts/check-commit-docs-exists.sh HEAD` → expect the normal verdict (0 or 1 on the merits of the tip commit's own message).

Integration

- Wired into `scripts/verify-all-constitution-rules.sh` as a standard gate (default scope: `HEAD~5..HEAD` to balance signal vs noise).
- Pre-push hooks transitively run via `scripts/check-constitution.sh`.
- Recommended pre-push scope: `@{u}..HEAD` to verify ALL unpushed commits before propagation.

Hermetic test

`tests/check-constitution/test_commit_docs_exists.sh` — 10 fixtures:

1. Commit with no path refs → pass
2. All-existing paths → pass
3. Orphan reference → reject (exit 1) + path listed
4. Stale ref inside `~~strikethrough~~` → pass (skipped)
5. Stale ref inside backticks → pass (skipped)
6. Fuzzy basename short-form → pass (resolved via `find`)
7. Indented Bluff-Audit/quoted-output content → pass (skipped)
8. Submodule-nested reference (no submodule prefix in prose) → pass (resolved via `git submodule foreach --recursive`)
9. Submodule-nested-LOOKING reference that resolves nowhere → reject (exit 1) — proves fixture 8 is not a blanket exemption
10. Real-repo HEAD sanity check → pass

Run:

```
bash tests/check-constitution/test_commit_docs_exists.sh
```

Classification: project-specific (the convention is universal per HelixConstitution; the path-prefix whitelist is Lava-specific).